

Design Choices and Implementation Summary

I developed two REST APIs using **FastAPI**. The first API translates PDF text and generates a new downloadable PDF, while the second API applies a watermark to every page. I used **PyMuPDF** for PDF text extraction, **FPDF2** for PDF generation, and **HarfBuzz** for proper Bangla text shaping. I also used a Bangla Unicode font, [Kalpurush.ttf](#), to ensure that Bengali characters are rendered correctly in the generated PDF.

For translation, I used a completely free multi-provider approach. **MyMemory** is used as the primary translator, and Google's public translation endpoint is used as a fallback when MyMemory fails. To improve the translation service, I added language-code normalisation, smart text chunking, request delays, timeout handling, limited retries, and a character safety limit. I also added an in-memory cache so that repeated text is not translated multiple times. These improvements make the service more efficient and reduce unnecessary API requests.

Compared with other project, my implementation improves reliability by adding caching, a strict character limit, better chunk splitting, timeout protection, and controlled retry behaviour. The existing API request format and Bangla PDF generation process were kept unchanged.

For watermarking, I used PDF drawing functions to place the selected text, colour, opacity, and position on every page. I also added automated tests for both translation and watermark APIs.

Known limitations are that MyMemory and Google public translation endpoints are free but have rate limits and may become temporarily unavailable. The cache is stored in memory, so it is cleared when the server restarts. Scanned PDFs may also require OCR, and complex PDF layouts such as tables, images, and diagrams may not be perfectly preserved during translation.